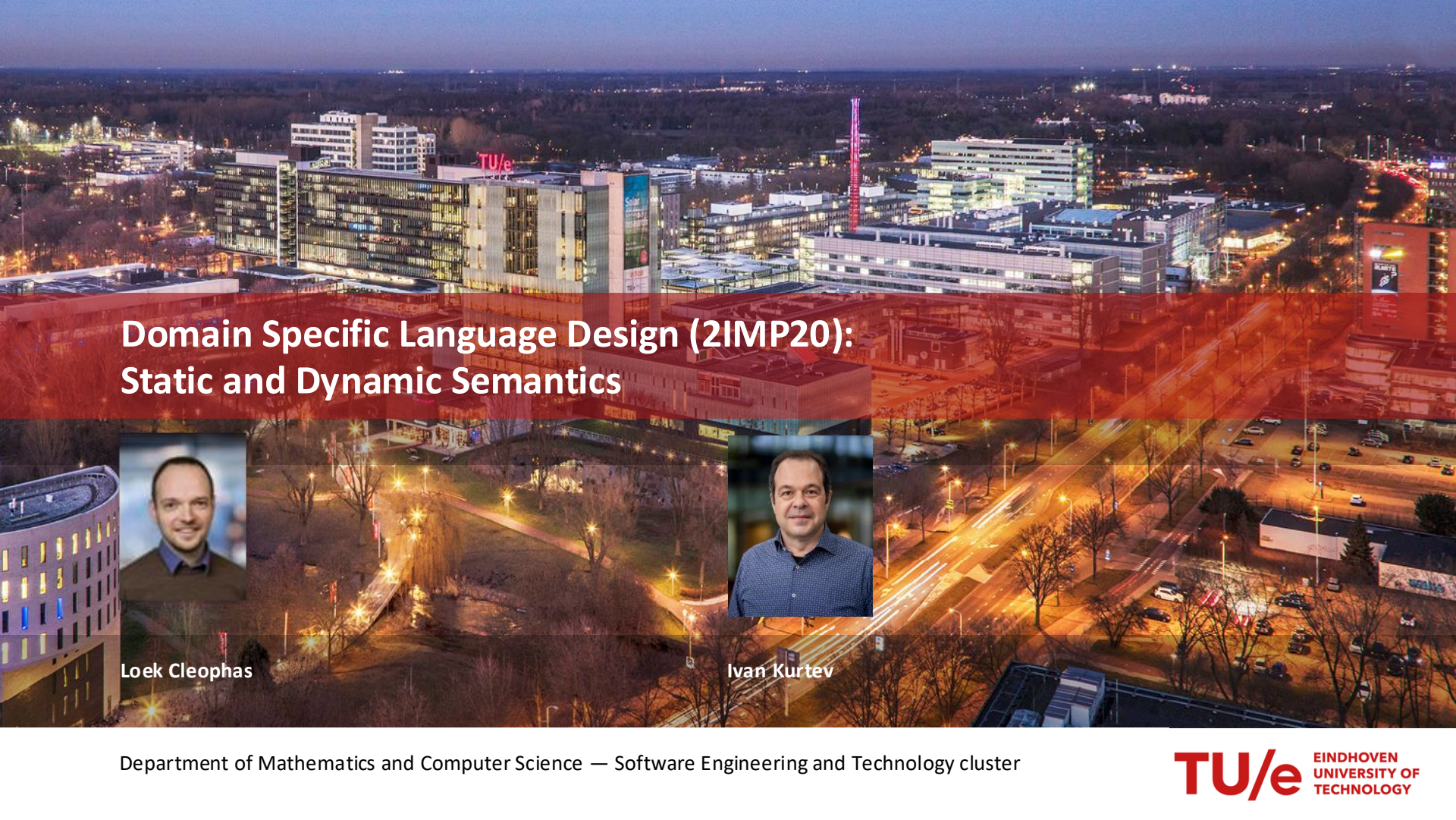




Domain Specific Language Design (2IMP20): Static and Dynamic Semantics



Loek Cleophas



Ivan Kurtev

Background information

Reading:

- Chapter 3, 4 and 5 of *Introduction to Compiler Design*
Covers scoping, interpretation, type checking
- Chapter 8 and 9 of *Software Languages*
Covers operational semantics + type checking

Implementation of a DSL

Definition of a (programming) language involves:

- abstract syntax
- concrete syntax:
 - textual syntax
 - graphical syntax
- semantics:
 - static
 - dynamic

Static semantics

- need for catching (trivial) errors before code generation
- necessity depends on the complexity of the language
- defines restrictions on syntactically valid programs:
 - cannot be expressed in context-free grammar
 - rules for:
 - scoping
 - identification of variables, functions, etc.
 - type systems
 - data/control flow
 - checked before executing a program

Over time: languages *have become stricter* with respect to *static semantics*

Static semantics – variable declarations

Languages like *Basic*, *Python*, *JavaScript* allowed **immediate use of variables without explicit declarations**

- misspelling in variable names led to unexpected behavior

Languages like *Java* **require an explicit declaration** of, for instance, a variable and its type

- an environment is needed to store variables and their types
- language specific issues to consider:
 - life-time of variables
 - scoping of variables

Static semantics

Life-time of a variable = time from *creation* (allocation)
to *destruction* (de-allocation)

- *Global variable's life-time is the program's run-time*
 - Variables that can be used anywhere in a program
- *Local variable's life-time is an activation of a block*
 - Only available within the block where they are declared
 - A *block* is a program construct that includes local declarations
 - An *activation* of a block is the time interval that the block is executed

For example, in a state machine-based language:

- *channels* could be global, and *variables* could be local

Static semantics

The *scope* of a declaration is the part of the program where the declaration is effective

A *block* is a language construct that delimits the scope of declarations within it

- monolithic block structure, e.g. Cobol
- flat block structure, e.g. Fortran
- nested block structure, e.g. Algol, C, Java, Python

Static semantics

Blocks:

- A *block command* is a form of command that contains a local declaration D and a subcommand C
 - In C/C++ and Java: `{ D C }`
`if (x > y) {int z = x; x = y; y = z;}`
 - z is not visible/alive outside this block
- A *block expression* is a form of expression that contains a local declaration D and a sub-expression E
 - Haskell provides block expressions: `let D in E`
 - D is only visible in E

Static semantics

When creating a type checker, you **need to deal with bindings by means of *symbol tables***

- If identifiers occur in an expression, such an expression cannot be understood in isolation:
 - its meaning depends on the declarations of these identifiers elsewhere in the program
- A *binding* is an association between an identifier and an entity such as value, variable or procedure, type
- A *symbol table* is a set of bindings:
 - consider the symbol table consisting of (variable name, type)

Static semantics

Symbol table/type environment for Pico in Rascal:

```
alias TENV = tuple[ map[PicoId, TYPE] symbols,  
                  list[tuple[loc l, str msg]] errors];  
  
TENV addError(TENV env, loc l, str msg) =  
    env[errors = env.errors + <l, msg>];
```

Pico language has no block structure: *single scope*!

Static semantics

Operations on symbol tables:

1. *Creation* of an *empty* symbol table
2. *Binding a name to a piece of information*. In case the name is already defined in the symbol table, the new binding takes precedence over the old, or an error message is to be generated
3. *Look up* a name in a symbol table to find the information the name is bound to. If the name is not defined in the symbol table, we need to be told that
4. *Entering a new scope* (requiring #1, creation!)
5. *Exiting a scope*, reestablishing the symbol table to what it was before the scope was entered

Static semantics

Symbol table/type environment **for block structured languages** in Rascal:

```
alias SENV = tuple[map[Id, TYPE] symbols];
alias TABLES = tuple[list[SENV] blocks];

TABLES enterBlock(TABLES tbls, SENV senv) = <push(senv, tbls.blocks)>;
TABLES leaveBlock(TABLES tbls) = <tail(tbls.blocks)>;
```

Per block a separate type environment is required

- Created when entering a block
- Destroyed when leaving a block

Finding a declared variable may involve inspecting all type environments

Static semantics

E.g. type,
variable/procedure name;
later for dyn. semantics:
value



Scope and visibility

- A *binding occurrence* of identifier $\mathbb{I}$ is an occurrence where $\mathbb{I}$ is bound to **entity** $\mathbb{X}$
- An *applied occurrence* of $\mathbb{I}$ is an occurrence where use is made of entity $\mathbb{X}$ to which $\mathbb{I}$ is bound
- Each applied occurrence of $\mathbb{I}$ should correspond to exactly one binding occurrence of $\mathbb{I}$
 - An identifier $\mathbb{I}$ may be defined in multiple blocks
- **Nested blocks, some outer block contains a declaration of $\mathbb{I}$:**
 - Inner block does not contain a declaration of $\mathbb{I}$ $\rightarrow$ declaration is visible throughout outer and inner blocks
 - Inner block contains a declaration of $\mathbb{I}$ $\rightarrow$ inner block declaration hides outer block declaration
 - This leads to “nested” symbol tables

Static semantics

Static vs dynamic scoping

- A language is **statically scoped** if the body of a procedure is executed **in the environment of the procedure's definition**
 - compile-time binding of identifiers
- A language is **dynamically scoped** if the body of a procedure is executed **in the environment of the procedure call**
 - run-time binding of identifiers

Nearly all programming languages (C, C++, Java, etc.) are statically scoped...

Static semantics

Static vs dynamic scoping

- Consider:

```
const int s = 2;  
int f(int x) { return s * x;}
```

```
void p(int y) { print(f(y));}  
p(3)
```

(1)

```
void q(int z) { const int s = 3; print(f(z));}  
q(3)
```

(2)

- What is the value printed at (1) and (2)?

Questions?



Static semantics — Declarations

A *declaration* is a language construct to produce a binding

Types of simple declarations:

- type
- constant
- variable
- procedure

Procedure and function definitions

A *procedure definition* binds an identifier to a procedure

- function
- procedure

Example function definition:

```
bool even(int n) {return (n % 2 == 0);}
```

Example procedure definition:

```
void double(int& n) { n *= 2; }
```

Procedure and function definitions

A *function/procedure* is an entity that embodies a computation

- a *function* embodies an expression to be evaluated
- a *procedure* embodies a command to be executed

Both implement **procedural abstraction**

Parameters and arguments

Parameters and arguments

- An *argument* is a value or other entity that is passed to a procedure
- An *actual parameter* is an expression that yields an argument
- A *formal parameter* is an identifier through which a procedure can access an argument
- Association between formal parameter and argument is called *parameter mechanism*
 - two basic concepts:
 - copy parameter mechanism
 - reference parameter mechanism

Parameters and arguments

Copy parameter mechanism allows the transfer of values to and from a procedure

- *Pass by value*
- A formal parameter FP denotes a local variable of the procedure

Reference parameter mechanism: A *reference parameter* allows for the formal parameter FP to be bound directly to the argument

- *Pass by reference*
- Modifications of the value may be visible outside the body of the procedure

Overloading

An identifier is *overloaded* if it denotes two or more distinct procedures in the same scope

- In many programming languages operators for certain built-in functions are overloaded
- “-” operator:
 - integer negation (`Integer` $\rightarrow$ `Integer`)
 - floating-point negation (`Float` $\rightarrow$ `Float`)
 - integer subtraction (`Integer` $\times$ `Integer` $\rightarrow$ `Integer`)
 - floating-point subtraction (`Float` $\times$ `Float` $\rightarrow$ `Float`)

Overloading

Identifier F denotes

- function (f_1) of type $S_1 \rightarrow T_1$
- function (f_2) of type $S_2 \rightarrow T_2$
- *context-independent overloading* requires S_1 and S_2 are non-equivalent
 - if actual parameter E of $F(E)$ is of type S_1 then F denotes f_1
 - if E is of type S_2 then F denotes f_2
- with context-independent overloading *the function can be uniquely identified by the type of the actual parameter*

Overloading

Identifier F denotes

- function (f_1) of type $S_1 \rightarrow T_1$
- function (f_2) of type $S_2 \rightarrow T_2$
- *context-dependent overloading* requires S_1 and S_2 are non-equivalent or T_1 and T_2 are non-equivalent
 - if S_1 and S_2 are non-equivalent, see previous slide
 - if S_1 and S_2 are equivalent, but T_1 and T_2 are non-equivalent the *context must be taken into consideration*
 - if the context $F(E)$ is of type T_1 then F denotes f_1
 - if the context is of type T_2 then F denotes f_2
- with context-dependent overloading, it is possible to formulate expressions which cannot be uniquely identified (How?)

Typing

Statically typed programming languages insist on *explicit declaration* of the type of entity in programs

- `integer I := E`
- In *Python* we can write a definition `I = E`
where type of `I` is not explicitly stated, but *inferred* from `E`
 - Similar: `var` declaration in recent Java versions (inferred compile time!)

Type inference is a process where the type of a declared entity is inferred instead of explicitly stated

Types and Data

What is the relation between data values and types?

- *Values* are entities manipulated by programs
- Different types of (programming) languages support different types of values
- Values are grouped into *types*
- A *type* is a **set of values with operations** that can be applied uniformly to all these values

Type Systems

A *type system* of a (programming) language is a set of rules

- Groups values into types
- Prevents illegal operations, like multiplication of strings by booleans: *type error*
- *Statically typed* language: each variable and expression has a fixed type
 - all operands can be type-checked at *compile-time*
- *Dynamically typed* language: values have fixed type, but variables and expressions have no fixed type.
 - operands can be only type-checked at *run-time*

Type Inferencing

Inference rules:

$$\frac{P_1 \quad \dots \quad P_n}{C} \quad [\perp]$$

where $P_1, \dots, P_n$ and C are *judgements* (*premises + consequent/conclusion*) and $\perp$ is simply a *label* used as reference.

If $n = 0$, the judgement is an *axiom*

$$C \quad [\perp]$$

Type Inferencing for Type Checking

Typing rules can be defined as inference rules

- Given the following context-free grammar:

syntax

```
Expr = "true" | "false" | "zero" |  
      "succ" Expr | "pred" Expr | "iszero" Expr |  
      "if" Expr "then" Expr "else" Expr
```

For type checking, *inference rules* as *typing rules* have to be defined

Type Inferencing – Specification by Inference Rules

The following typing rules as inference rules must be defined

true : booltype [true]

false : booltype [false]

zero : nattype [zero]

$$\frac{e : \text{nattype}}{\text{succ}(e) : \text{nattype}} \quad [\text{succ}]$$

$$\frac{e : \text{nattype}}{\text{pred}(e) : \text{nattype}} \quad [\text{pred}]$$

$$\frac{e : \text{nattype}}{\text{iszero}(e) : \text{booltype}} \quad [\text{iszero}]$$

$$\frac{e_0 : \text{booltype} \quad e_1 : T \quad e_2 : T}{\text{if}(e_0, e_1, e_2) : T} \quad [\text{if}]$$

A type checker for PICO

- The types are **natural** and **string**
- All variables should be declared before use
- Lhs and RhS of assignment should have equal type
- The test in while and if-then should be natural
- Operands of + and – should be natural; result is natural
- Operands of | | should be string; result string

Types

Variables

Language
constructs:

- statements
- operators

Syntax directed definitions

- For every construct in the abstract syntax
define a rule to describe
the typing/semantics/translation of that language construct
- Here: **type inference/checking**
- Later: **dynamic semantics**

Recall PICO abstract syntax

```
public data PROGRAM =  
    program(list[DECL] decls, list[STATEMENT] stats);  
  
public data DECL =  
    decl(PicoId name, TYPE tp);  
  
public data EXP =  
    id(PicoId name)  
    | natCon(int iVal)  
    | strCon(str sVal)  
    | add(EXP left, EXP right)  
    | sub(EXP left, EXP right)  
    | conc(EXP left, EXP right);  
  
public data STATEMENT =  
    asgStat(PicoId name, EXP exp)  
    | ifElseStat(EXP exp, list[STATEMENT] thenpart,  
                list[STATEMENT] elsepart)  
    | whileStat(EXP exp, list[STATEMENT] body);
```

Some preliminaries: type environment and error handling

```
alias TENV = tuple[ map[PicoId, TYPE] symbols,  
                    list[tuple[loc 1, str msg]] errors];  
  
TENV addError(TENV env, loc 1, str msg) =  
    env[errors = env.errors + <1, msg>];  
  
str required(TYPE t, str got) = "Required <getName(t)>, got <got>";  
str required(TYPE t1, TYPE t2) = required(t1, getName(t2));
```

A type checker for PICO

Checking basic expressions

```
TENV checkExp(exp:natCon(int N), TYPE req, TENV env)=
  req == natural() ? env :
    addError(env, exp@location, required(req, "natural"));

TENV checkExp(exp:strCon(str S), TYPE req, TENV env) =
  req == string() ? env :
    addError(env, exp@location, required(req, "string"));

TENV checkExp(exp:id(PicoId Id), TYPE req, TENV env){
  if(!env.symbols[Id]?){
    return addError(env, exp@location, "Undeclared variable <Id>");
  }
  tpid = env.symbols[Id];
  return req == tpid ? env :
    addError(env, exp@location, required(req, tpid));
}
```

A type checker for PICO

Checking binary expressions

```
TENV checkExp(exp:add(EXP E1, EXP E2), TYPE req, TENV env) =  
  req == natural() ? checkExp(E1, natural(), checkExp(E2, natural(), env))  
                  : addError(env, exp@location, required(req, "natural"));  
  
TENV checkExp(exp:sub(EXP E1, EXP E2), TYPE req, TENV env) =  
  req == natural() ? checkExp(E1, natural(), checkExp(E2, natural(), env))  
                  : addError(env, exp@location, required(req, "natural"));  
  
TENV checkExp(exp:conc(EXP E1, EXP E2), TYPE req, TENV env) =  
  req == string() ? checkExp(E1, string(), checkExp(E2, string(), env))  
                  : addError(env, exp@location, required(req, "string"));
```

A type checker for PICO

Checking simple statements

```
TENV checkStat(stat:asgStat(PicoId Id, EXP Exp), TENV env) {  
  if(!env.symbols[Id])  
    return addError(env, stat@location, "Undeclared variable <Id>");  
  tpid = env.symbols[Id];  
  return checkExp(Exp, tpid, env);  
}  
  
TENV checkStat(stat:ifElseStat(EXP Exp,  
  list[STATEMENT] Stats1, list[STATEMENT] Stats2),  
  TENV env) {  
  env0 = checkExp(Exp, natural(), env);  
  env1 = checkStats(Stats1, env0);  
  env2 = checkStats(Stats2, env1);  
  return env2;  
}
```

A type checker for PICO

Checking simple statements

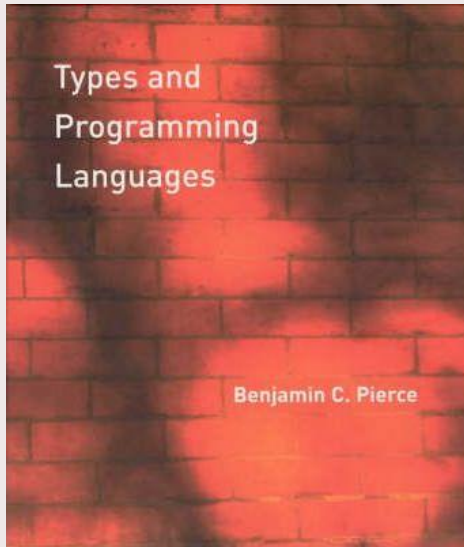
```
TENV checkStat(stat:whileStat(EXP Exp,  
    list[STATEMENT] Stats1),  
    TENV env){  
    env0 = checkExp(Exp, natural(), env);  
    env1 = checkStats(Stats1, env0);  
    return env1;  
}  
  
TENV checkStats(list[STATEMENT] Stats1, TENV env){  
    for(S <- Stats1){  
        env = checkStat(S, env);  
    }  
    return env;  
}
```

A type checker for PICO

Checking declarations and a program

```
TENV checkDecls(list[DECL] Decls)=  
  <(Id:tp | decl(PicoId Id, TYPE tp)<- Decls), []>;  
  
public TENV checkProgram(PROGRAM P){  
  if(program(list[DECL] Decls, list[STATEMENT] Series) := P){  
    TENV env = checkDecls(Decls);  
    return checkStats(Series, env);  
  } else  
    throw "Cannot happen";  
}
```

The typing bible



Questions?



Lookahead

- 2026/05/29: more semantics: dynamic semantics
- 2026/06/03: Ivan (code generation)
- 2026/06/05: DSL design (the bigger picture)
- 2026/06/10: guest lecture Eugen Schindler, Canon Production printing
- 2026/06/12: guest lecture, tbd
- 2026/06/17 and 2025/06/19: backup slots